Formal Languages and Compilers

Lab6

Numpy, Matplotlib, Seaborn, Streamlit in Python

Useful Python packages for Data Analysis

- Numerical Computing:
  - numpy
  - scipy
  - statsmodels
- Data Vizualisation:
  - matplotlib
  - seaborn
  - plotly
  - altair
- Data manipulation:
  - pandas
  - polars
- Machine Learning:
  - sckit-learn
  - keras
- Web app development for data vizualisation: Streamlit

numpy

- Open-source library, used for scientific computing and working with arrays (*pip install numpy*)
- Arrays need to be declared, can store data compactly and are great for numerical operations
- You can consider 2-D arrays as a table with rows and columns, with the dimension standing in for the row and the index for the column.
- You can iterate through arrays, split them, sort them and many more
- random() can be used to generate random populated arrays of different types
- Simple example

```
import numpy as np

arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
#2D array
arr2 = np.array([[0, 1, 2, 3, 4, 5],
                 [6, 7, 8, 9, 10, 11]])

#print the values in index1, index2, index3, index4
print(arr[1:5])
print("----")
#print every other number in the array
print(arr[::2])
print("----")
#print from both parts of arr2
print(arr2[0:2, 2:5])
print("----")

x=np.random.random(2)
print(x)
```

```
[1 2 3 4]
----
[ 0  2  4  6  8 10]
----
[[ 2  3  4]
 [ 8  9 10]]
----
[0.13262625 0.60643364]
```

numpy
-examples-

```python
import numpy as np

arr=np.array([0,1,2,3,4,5,6,7,8,9,10,11])
arr3=np.array([50,40,20,30,60,80,70])
arr2=np.array([[0,1,2,3,4,5],[6,7,8,9,10,11]])

#INDEX
#access element from 1st row, third column
print("1st row, 3rd column:", arr2[0,2])
#access the 4th element from second array
print("2nd row, 4th column:", arr2[1,3])

#array shape=no. of elements from the same dimension
print("Aray shape: ", arr2.shape)
#Using reshape, we can adjust the amount of elements
#in each dimension or add/remove dimensions.
print("Array reshape to 2D: ", arr.reshape(3,4))
#concatanate arrays
print("New array is: ", np.concatenate((arr,arr3)))
#splt array
print("List with splitted arrays: ", np.array_split(arr, 4))
print("Third array in the list: ", np.array_split(arr, 4)[2])
print("Split 2D array: ", np.array_split(arr2, 2))
#Search by index inside array
print("Index of elements divisible by 5:",  np.where(arr3%4 == 0))
#Sort array
print("Sorted array: ", np.sort(arr3))
```

```
Console 1/A

In [30]: runfile('D:/Facultate/Predat/Python/Lab1.py', wdir='D:/Facultate/Predat/Python')
1st row, 3rd column: 2
2nd row, 4th column: 9
Aray shape:  (2, 6)
Array reshape to 2D:  [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
New array is:  [ 0  1  2  3  4  5  6  7  8  9 10 11 50 40 20 30 60 80 70]
List with splitted arrays:  [array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8]), array([ 9, 10, 11])]
Third array in the list:  [6 7 8]
Split 2D array:  [array([[0, 1, 2, 3, 4, 5]]), array([[ 6,  7,  8,  9, 10, 11]])]
Index of elements divisible by 5: (array([1, 2, 4, 5], dtype=int64),)
Sorted array:  [20 30 40 50 60 70 80]

In [31]:
```

numpy
-useful functions for arrays-

- zeros() – create an array populated with 0
- ones() – create an array populated with 1
- arrange() – return an array with values within a specified range
- identity() – create an identity matrix (m x m matrix with 1s in main diagonal)
- min() – return the minimum value within an array
- max() – return the maximum value within an array
- mean() – return the mean of the elements within an array
- put() - replace specified elements of an array with given values
- copyto() - copy the content of one array into another.
- interesctld() - return all the unique values from both arrays in a sorted manner
- setdiffld() - return all the unique elements from array 1 that are not present in array 2
- unionld() – combine 2 arrays into one (no duplicates)
- savetxt() - save the content of an array inside a text file
- loadtxt() - load the content of an array from a text file

matplotlib

- Open-source library used for plotting graphs (pip install matplotlib)
- pyplot – module inside matplotlib with a large variety of functions

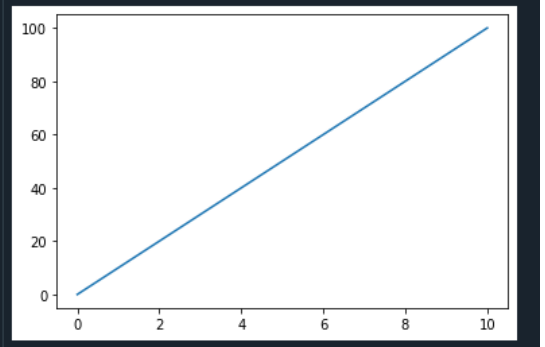
```
import matplotlib.pyplot as plt
import numpy as np

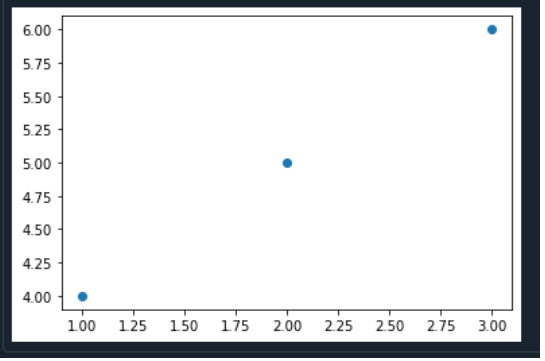

#horizontal axis
x=np.array([0,10])
#vertical axis
y=np.array([0,100])

plt.plot(x,y)
plt.show()


x1=np.array([1,2,3])
y1=np.array([4,5,6])

#plot points defined by x1 and y1
#o is called a marker and is used to emphasize points
#there are multiple types of markers besides o
plt.plot(x1,y1, "o")
plt.show()
```

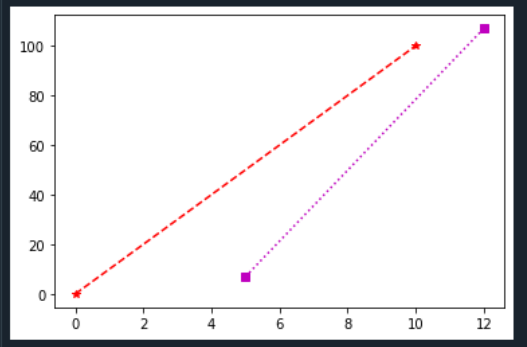


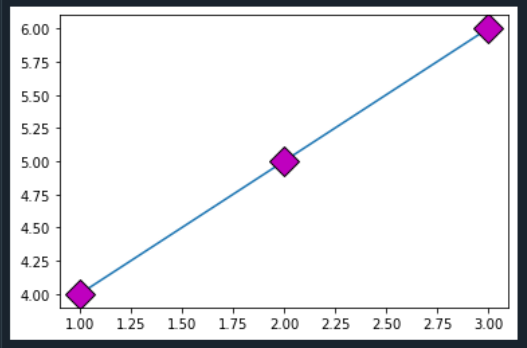



```
x2=np.array([5,12])
y2=np.array([7,107])

#marker line reference color
#marker * line reference -- color r
#points will be star-shaped, the line between them
#will be dashed and the color of both will be red
plt.plot(x,y,"*--r")
#add a new line in the plot
plt.plot(x2,y2,"s:m")
plt.show()

#marker-marker type
#mec-marker exterior color
#mfc-marker inside color
#ms-marker size
plt.plot(x1,y1, marker="D", mec="k", mfc = "m", ms=15)
plt.show()
```





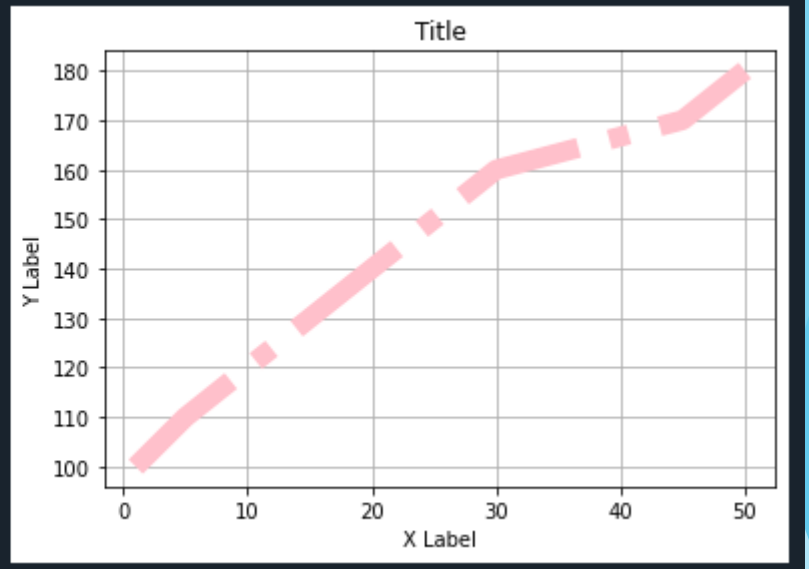
Examples

```
x2 = np.array([1,5,10,15,20,25,30,45,50])
y2 = np.array([100,110,120,130,140,150,160,170,180])

#line: linestyle-dash dot, color, width
plt.plot(x2, y2, linestyle="-.", color="pink", linewidth="10")
plt.grid()

plt.title("Title")
plt.xlabel("X Label")
plt.ylabel("Y Label")

plt.show()
```

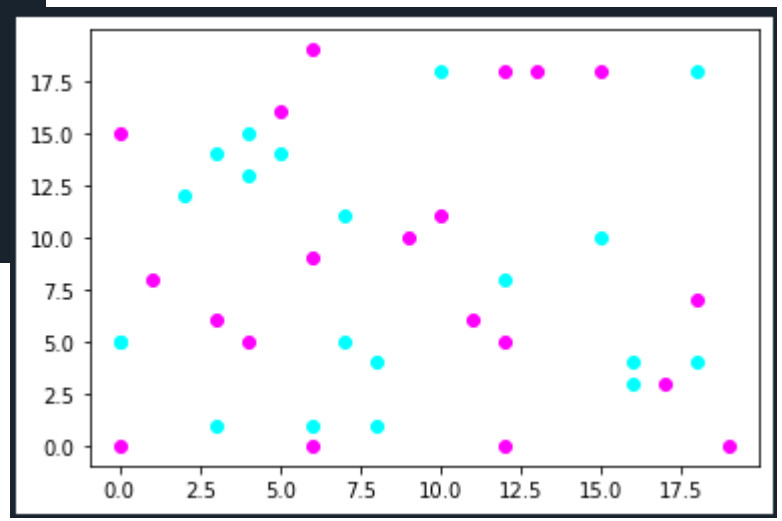



```
#scatter()-plot one dot for each point

x1 = np.random.randint(20, size=(20))
y1 = np.random.randint(20, size=(20))
plt.scatter(x1, y1, color = 'magenta')

x2 = np.random.randint(20, size=(20))
y2 = np.random.randint(20, size=(20))
plt.scatter(x2, y2, color = '#00FFFF')

plt.show()
```

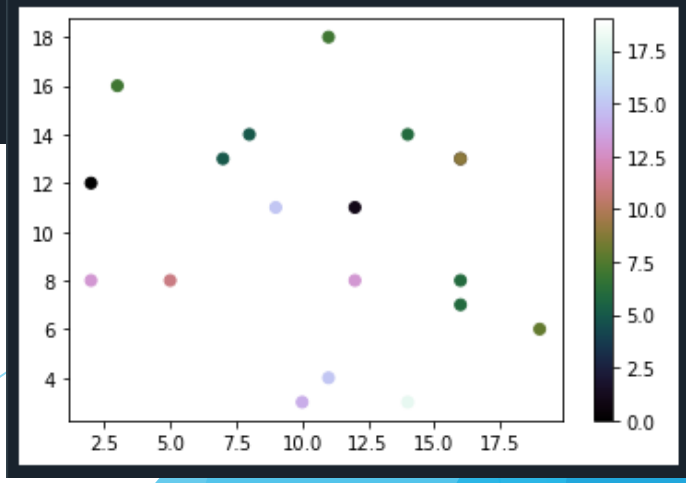



```
x3 = np.random.randint(20, size=(20))
y3 = np.random.randint(20, size=(20))
colors = np.random.randint(20, size=(20))

#adding a color map (list of colors, where each color has a value)
plt.scatter(x3, y3, c=colors, cmap='cubehelix')

plt.colorbar()

plt.show()
```



Example –subplot()

```
#subplot(no_rows, no_columns, graph_no)

x = np.array([0, 1, 2])
y = np.array([3, 4, 5])

plt.subplot(2, 3, 1)
plt.plot(x,y)
plt.title("Title1")

x = np.array([6, 7 ,8])
y = np.array([9, 10, 11])

plt.subplot(2, 3, 2)
plt.plot(x,y)
plt.title("Title2")

x = np.array([12, 14, 16])
y = np.array([19, 22, 25])

plt.subplot(2, 3, 3)
plt.plot(x,y)
plt.title("Title3")

x = np.array([26, 29, 32])
y = np.array([34, 36, 38])

plt.subplot(2, 3, 4)
plt.plot(x,y)
plt.title("Title4")

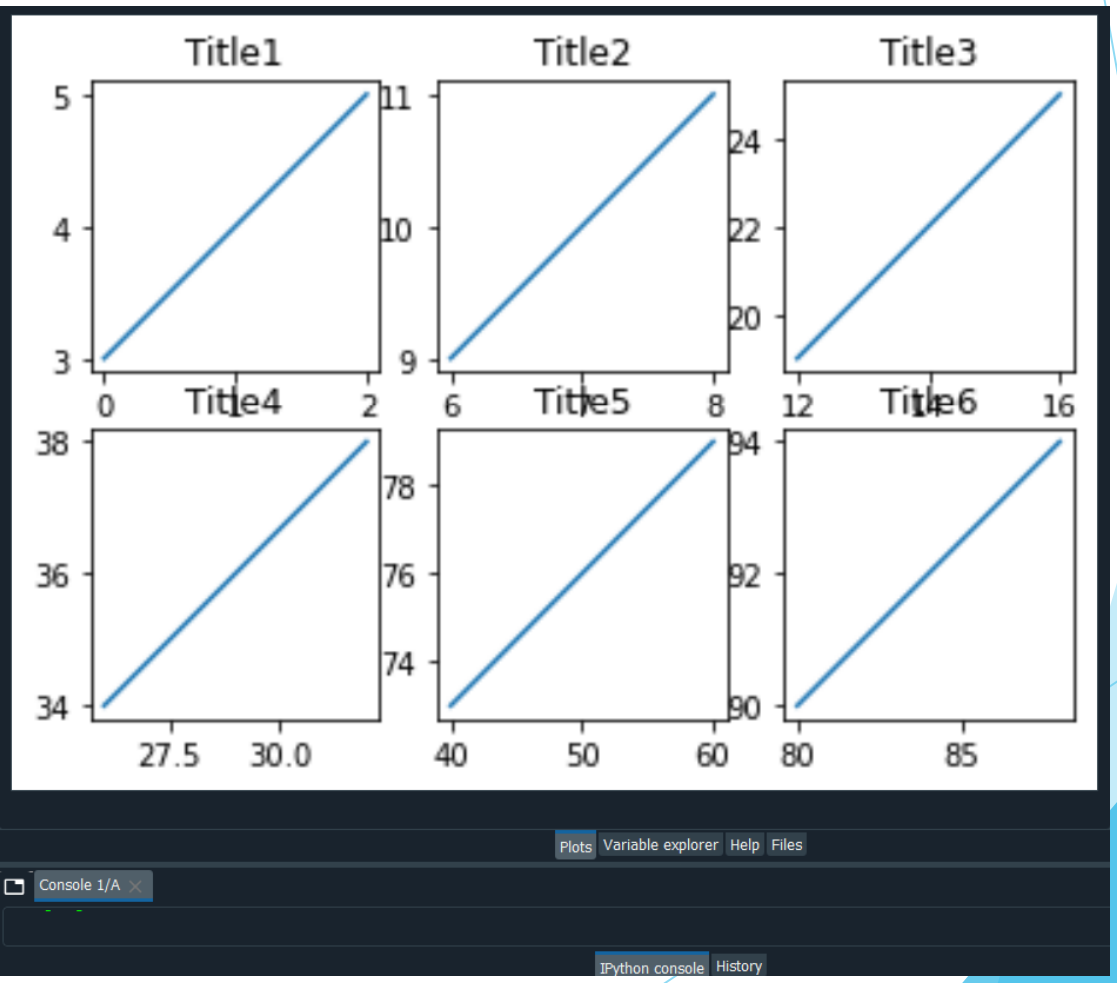
x = np.array([40, 50, 60])
y = np.array([73, 76, 79])

plt.subplot(2, 3, 5)
plt.plot(x,y)
plt.title("Title5")

x = np.array([80, 84, 88])
y = np.array([90, 92, 94])

plt.subplot(2, 3, 6)
plt.plot(x,y)
plt.title("Title6")

plt.show()
```



Examples - bars

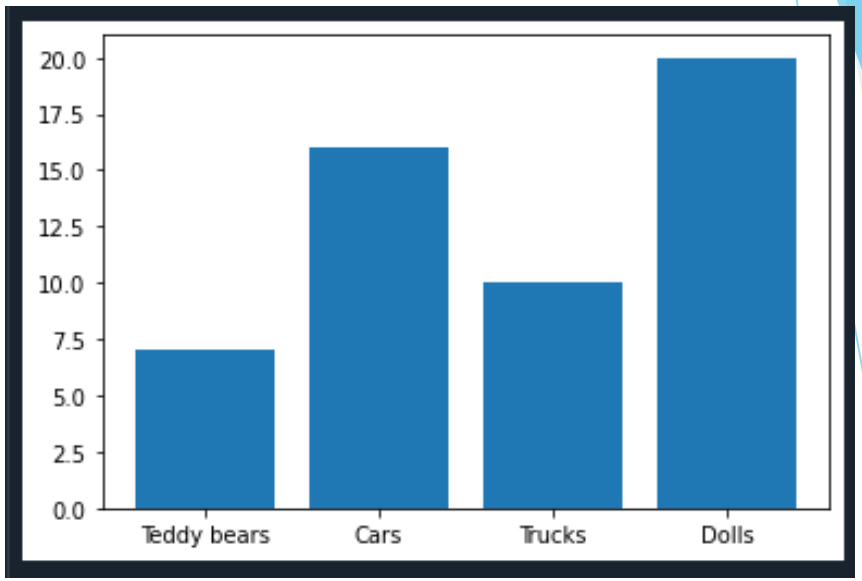
```
x = np.array(["Teddy bears", "Cars", "Trucks", "Dolls"])
y = np.array([7, 16, 10, 20])

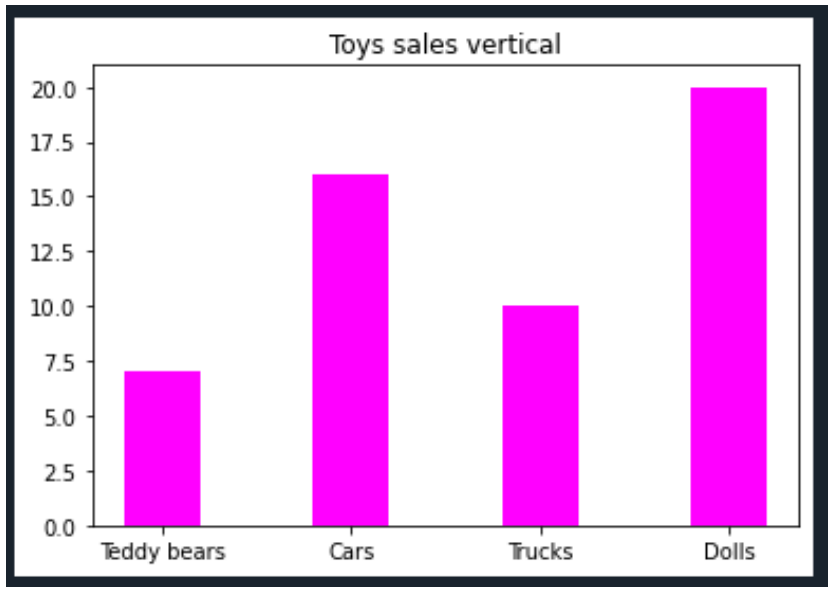
plt.bar(x, y)
plt.show()

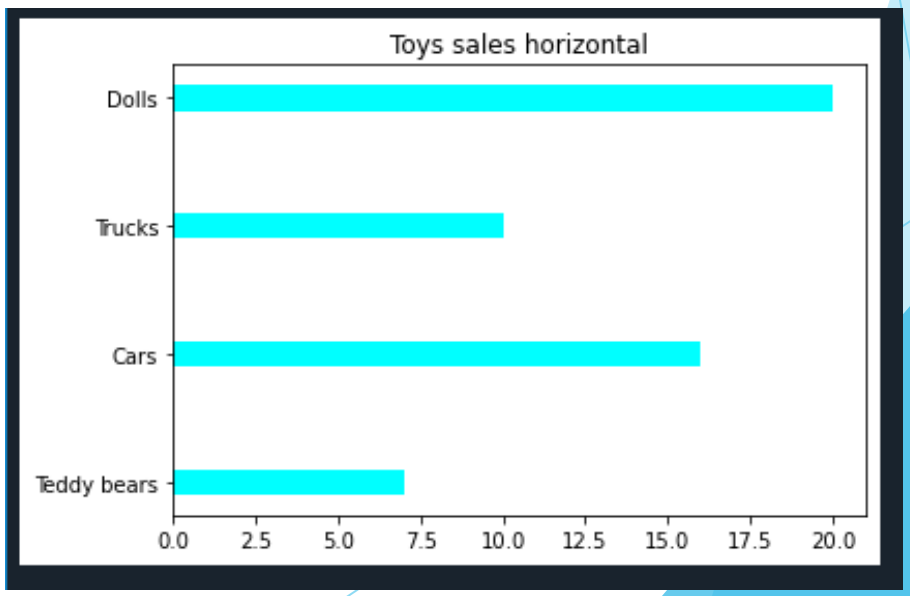
plt.bar(x, y, color = "magenta", width=0.4)
plt.title("Toys sales vertical")
plt.show()

plt.barh(x,y, color = "cyan", height=0.2)
plt.title("Toys sales horizontal")

plt.show()
```







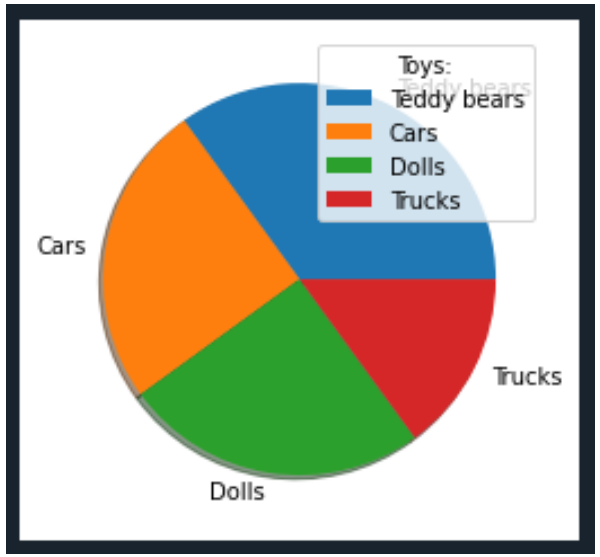
Examples - pies

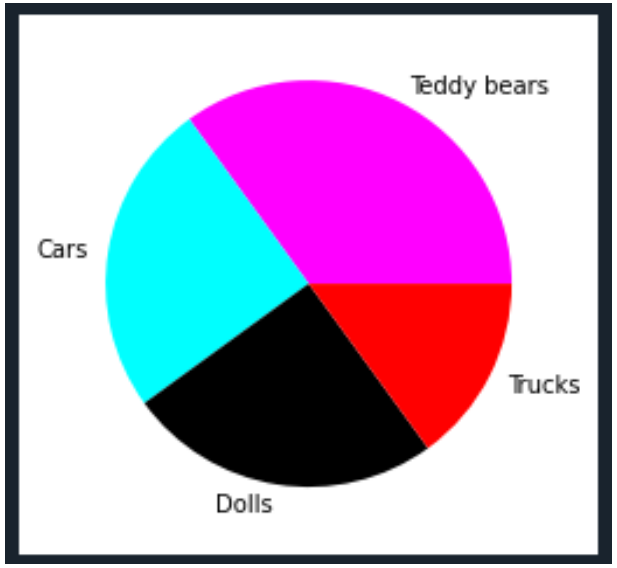
```
y = np.array([35, 25, 25, 15])
mylabels = ["Teddy bears", "Cars", "Dolls", "Trucks"]
mycolors=["magenta", "cyan", "black", "red"]
myexplode=[0,0.3,0,0.1]

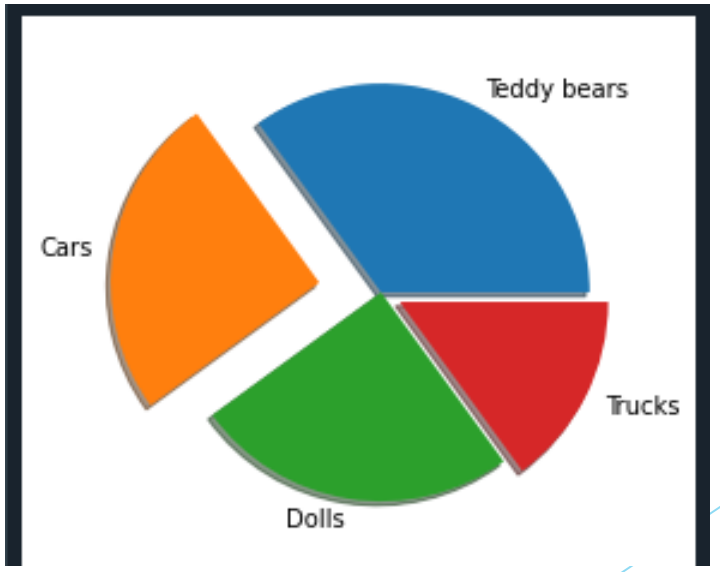
plt.pie(y, labels = mylabels, shadow=True)
plt.legend(title = "Toys:")
plt.show()

plt.pie(y, labels = mylabels, colors=mycolors)
plt.show()

plt.pie(y, labels = mylabels, explode=myexplode, shadow=True)
plt.show()
```







Seaborn

- Used for creating statistical visualizations (*pip install seaborn*)
- Built on top of matplotlib and integrated with pandas data structures
- Useful functions-axes level (work with plt.subplots()):
  - Lineplot(): draw a line plot () (display data points connected by straight lines)
  - scatterplot(): draw a scatter plot (used to display data points)
  - stripplot(): draw a categorical scatterplot using jitter to reduce overplotting
  - histplot(): draw a histogram (used to display the distribution of one or more variables)
  - boxplot(): draw a box plot (used to display distributions with respect to categories)
  - violinplot(): draw a violin plot (used to display distribution of data points after grouping by one/more variables, using KDEs-how data points are spread out)
  - barplot(): draw a bar plot (display point estimates and errors)
- Useful functions-figure level (good when needing one plot split by a category):
  - displot(): draw distribution plots in a FacetGrid (plot histograms, KDEs (kernel density estimate), etc)
  - relplot(): draw relational plots in a grid (plot scatter, line)
  - catplot(): draw categorical plots in a grid (plot Box, Violin, Bar, Strip, etc)
  - pairplot(): draw pairwise relationships in a dataset (plot histograms, KDEs, scatter)
  - heatmap(): plot rectangular data as a color-encoded matrix

Seaborn – example

```python
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")

print("First 5 rows")
print(iris.head())
print("\nShape:", iris.shape)                    # (150, 5)
print("\nSpecies counts ")
print(iris["species"].value_counts())
print("\nSummary statistics ")
print(iris.describe())
print("\n")
print(iris.groupby("species").describe().T)

```

```
First 5 rows
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Shape: (150, 5)

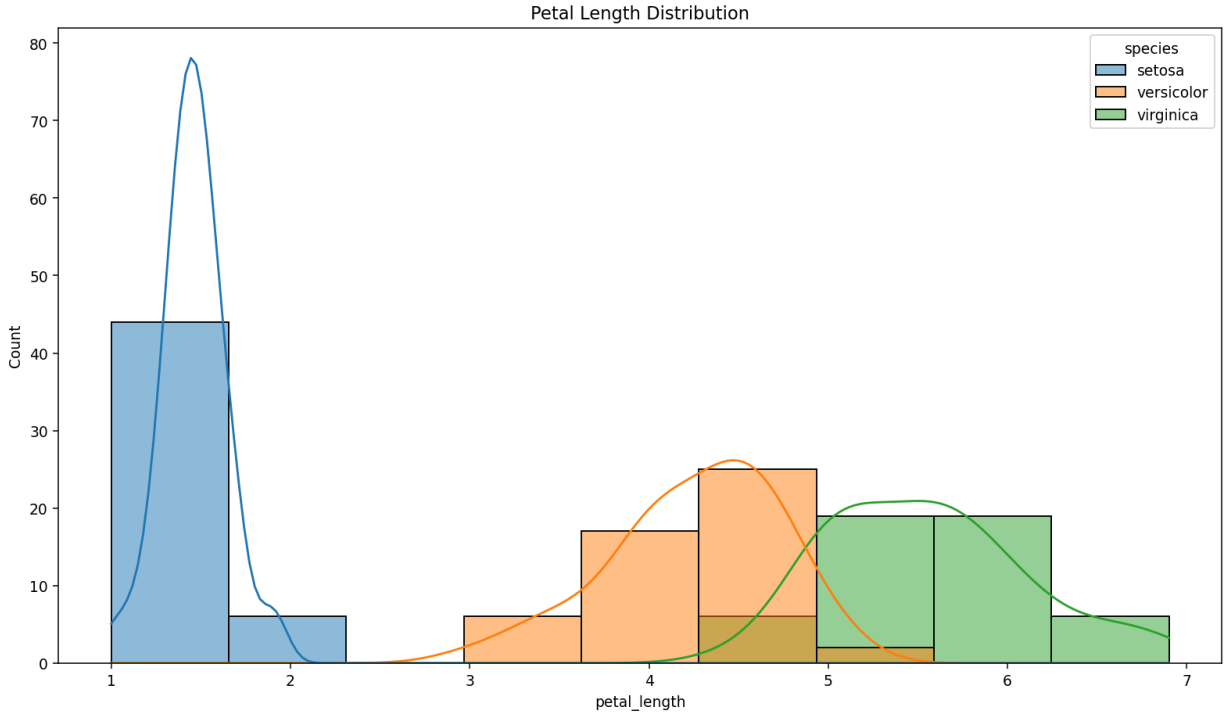
Species counts
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

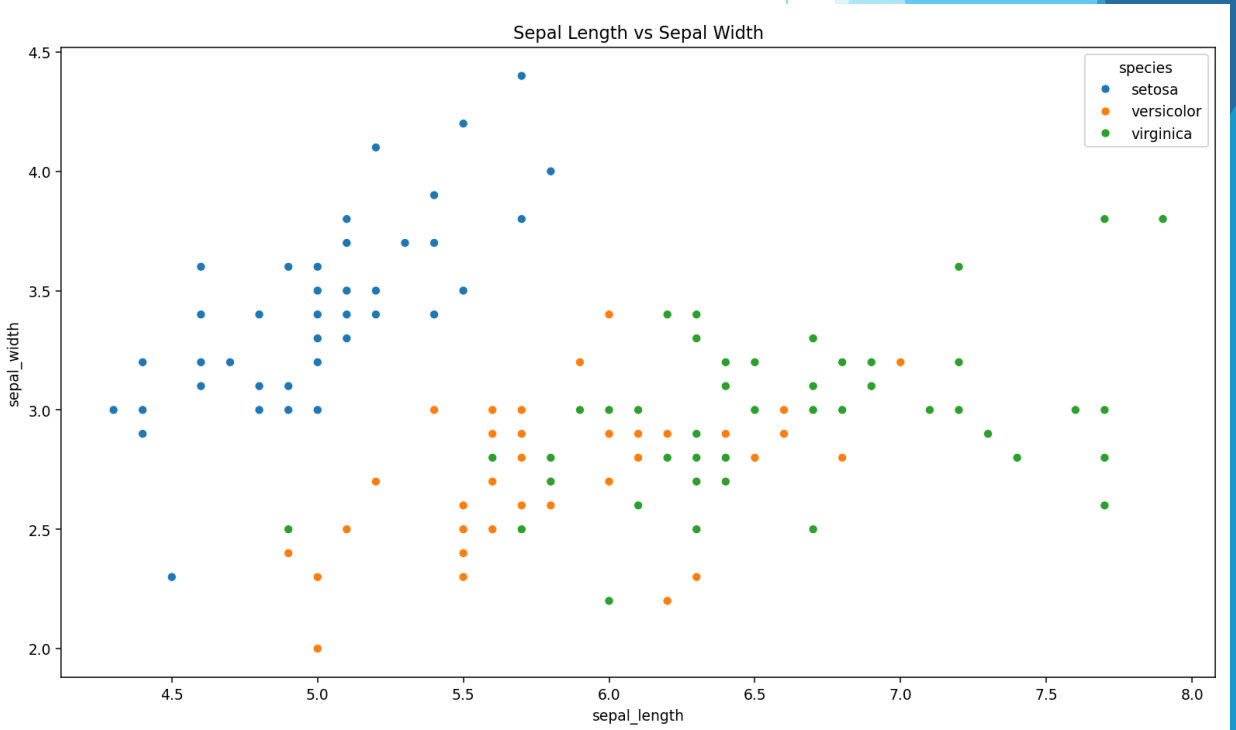
```
Summary statistics
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

```
species                setosa  versicolor  virginica
sepal_length count   50.000000   50.000000  50.000000
             mean     5.006000    5.936000   6.588000
             std      0.352490    0.516171   0.635880
             min      4.300000    4.900000   4.900000
             25%      4.800000    5.600000   6.225000
             50%      5.000000    5.900000   6.500000
             75%      5.200000    6.300000   6.900000
             max      5.800000    7.000000   7.900000
sepal_width  count   50.000000   50.000000  50.000000
             mean     3.428000    2.770000   2.974000
             std      0.379064    0.313798   0.322497
             min      2.300000    2.000000   2.200000
             25%      3.200000    2.525000   2.800000
             50%      3.400000    2.800000   3.000000
             75%      3.675000    3.000000   3.175000
             max      4.400000    3.400000   3.800000
petal_length count   50.000000   50.000000  50.000000
             mean     1.462000    4.260000   5.552000
             std      0.173664    0.469911   0.551895
             min      1.000000    3.000000   4.500000
             25%      1.400000    4.000000   5.100000
             50%      1.500000    4.350000   5.550000
             75%      1.575000    4.600000   5.875000
             max      1.900000    5.100000   6.900000
petal_width  count   50.000000   50.000000  50.000000
             mean     0.246000    1.326000   2.026000
             std      0.105386    0.197753   0.274650
             min      0.100000    1.000000   1.400000
             25%      0.200000    1.200000   1.800000
             50%      0.200000    1.300000   2.000000
             75%      0.300000    1.500000   2.300000
             max      0.600000    1.800000   2.500000
```

Seaborn – example

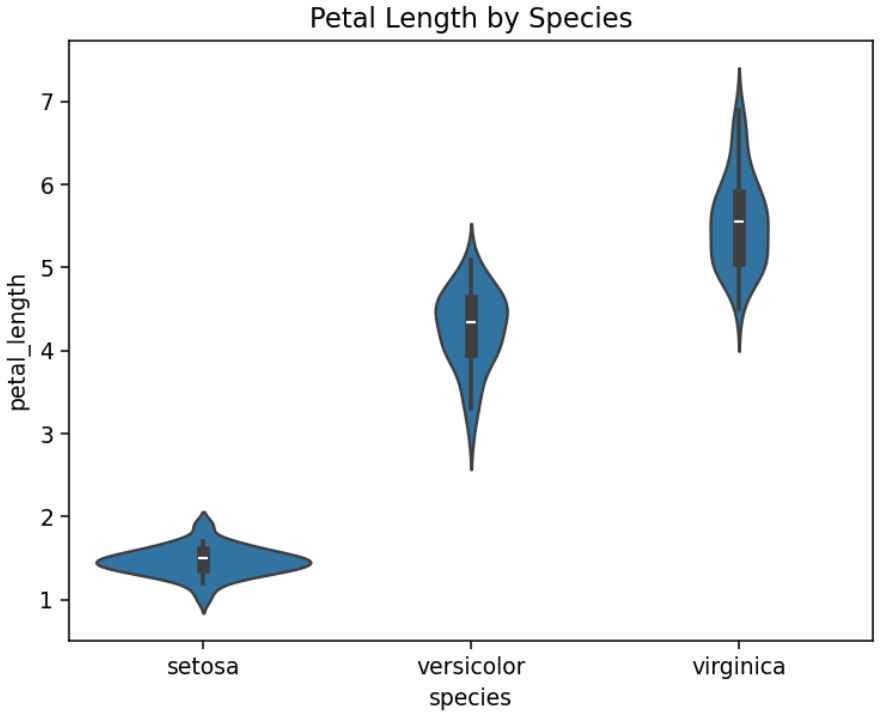
```
17  # Histogram with density curve, one color per species
18  sns.histplot(data=iris, x="petal_length", hue="species", kde=True)
19  plt.title("Petal Length Distribution")
20  plt.show()
21
22  # Scatter plot of two features
23  sns.scatterplot(data=iris, x="sepal_length", y="sepal_width", hue="species")
24  plt.title("Sepal Length vs Sepal Width")
25  plt.show()
26
```

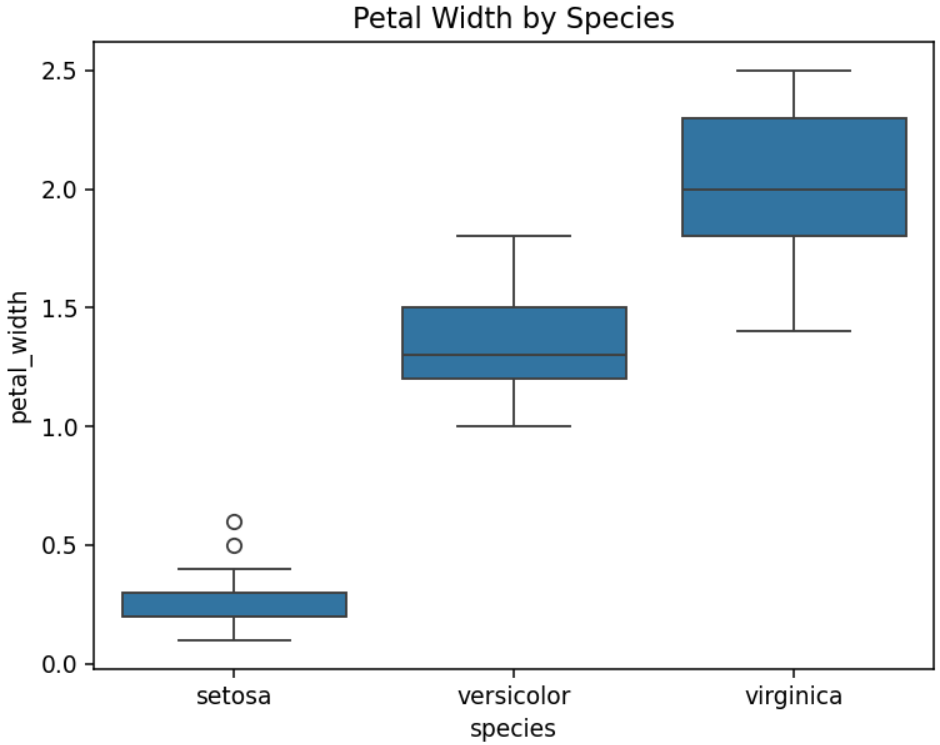


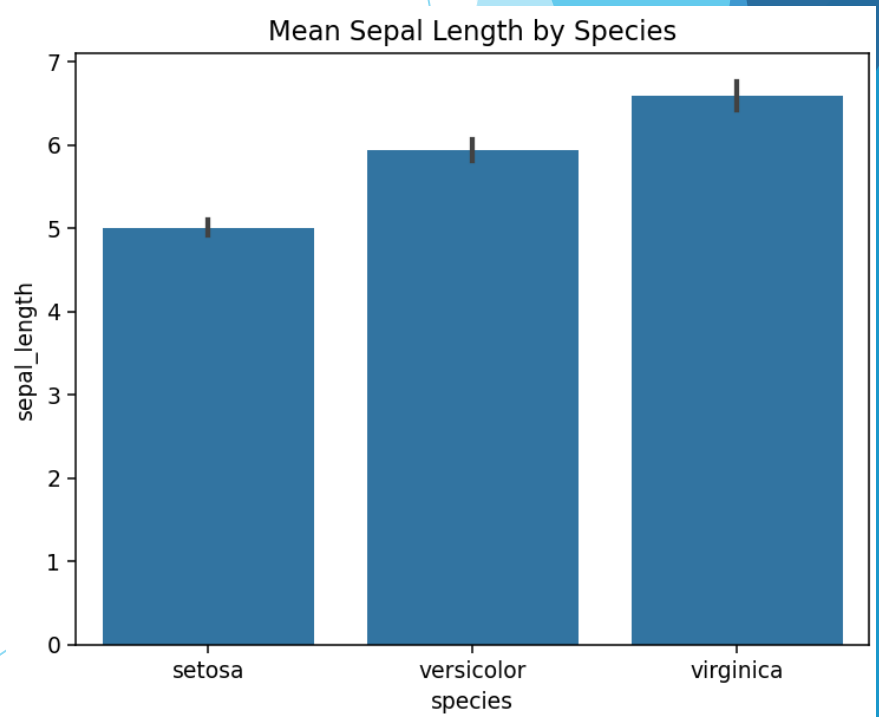


Seaborn - example

```
27  # Box plot: spread, median and outliers per species
28  sns.boxplot(data=iris, x="species", y="petal_width")
29  plt.title("Petal Width by Species")
30  plt.show()
31
32  # Bar plot: mean value per species
33  sns.barplot(data=iris, x="species", y="sepal_length")
34  plt.title("Mean Sepal Length by Species")
35  plt.show()
36
37  # Violin plot: box plot + density shape
38  sns.violinplot(data=iris, x="species", y="petal_length")
39  plt.title("Petal Length by Species")
40  plt.show()
41
```

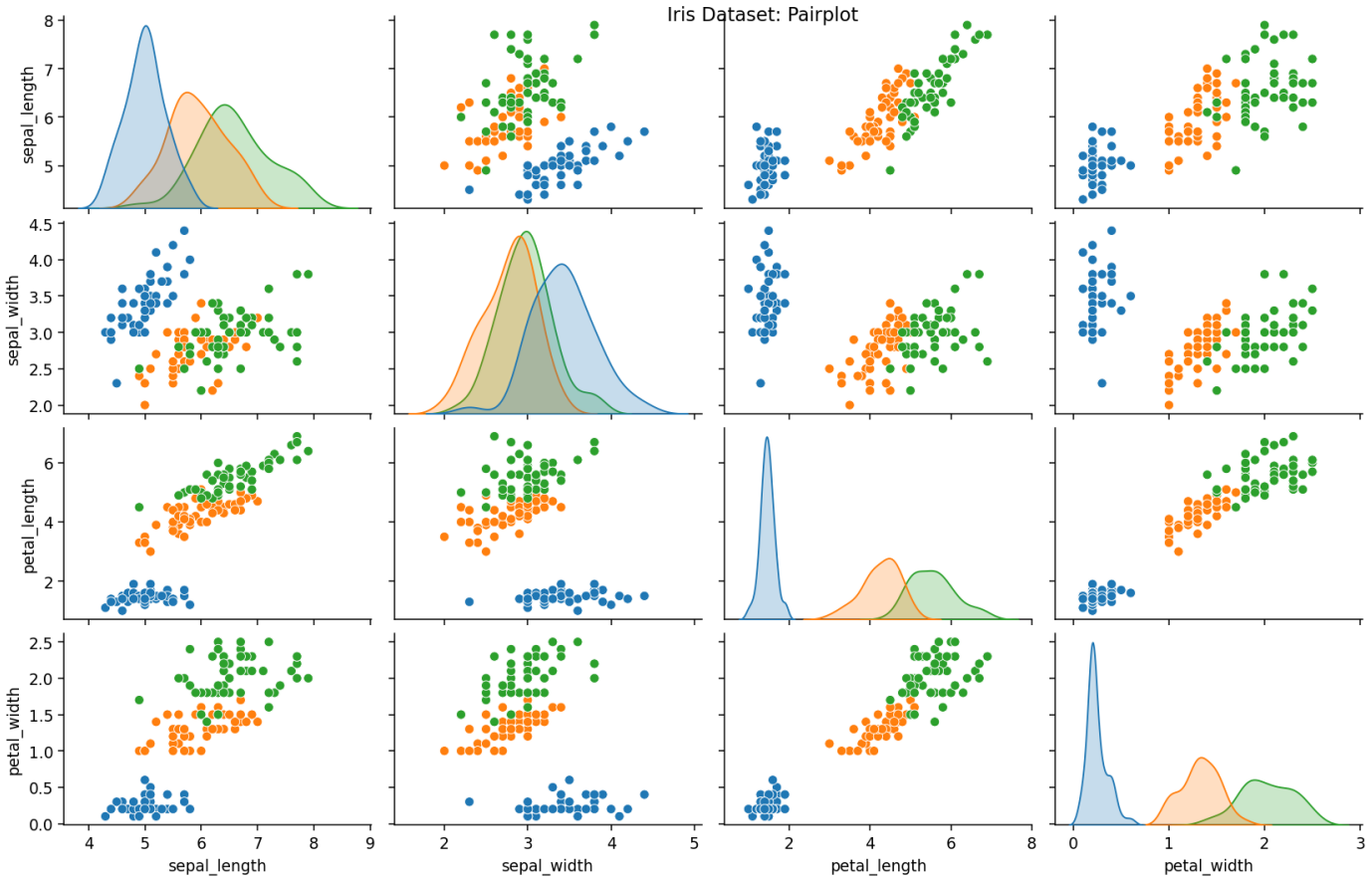


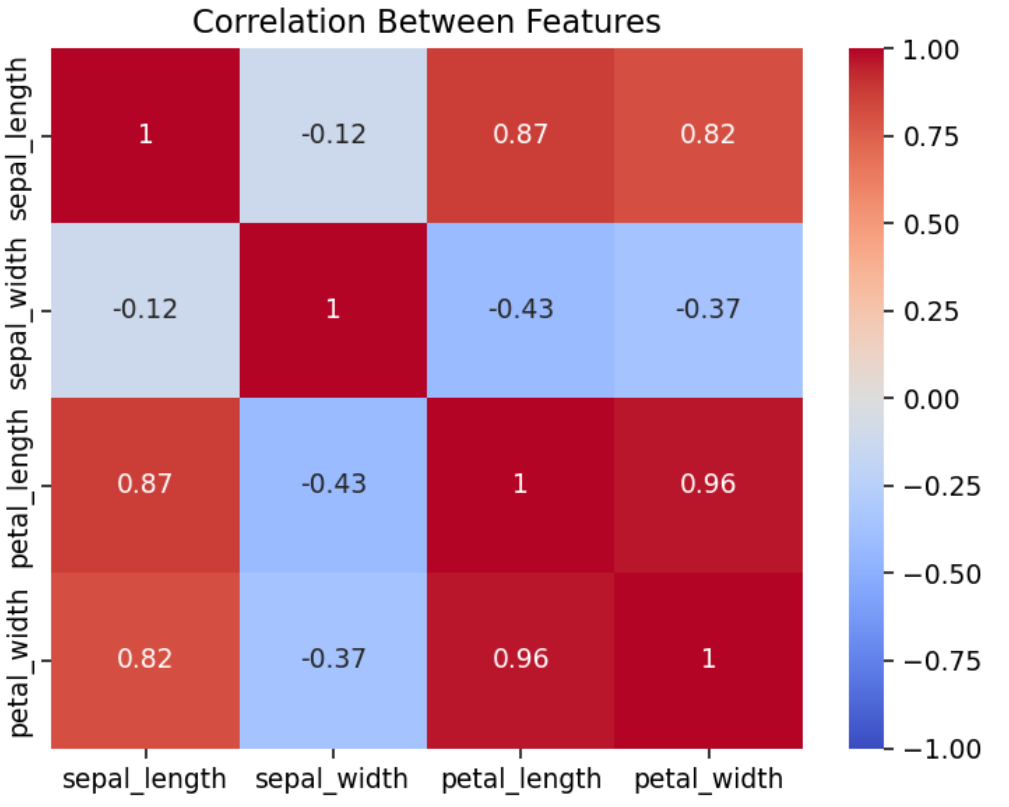




Seaborn – example

```
42
43  # Pairplot: every feature against every other
44  g = sns.pairplot(iris, hue="species")
45  g.figure.suptitle("\nIris Dataset: Pairplot", y=1.02)
46  plt.show()
47
48  # Correlation heatmap (numeric columns only)
49  corr = iris.drop(columns="species").corr()
50  ax = sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
51  ax.set_title("Correlation Between Features")
52  plt.show()
```





Streamlit

- pip install streamlit

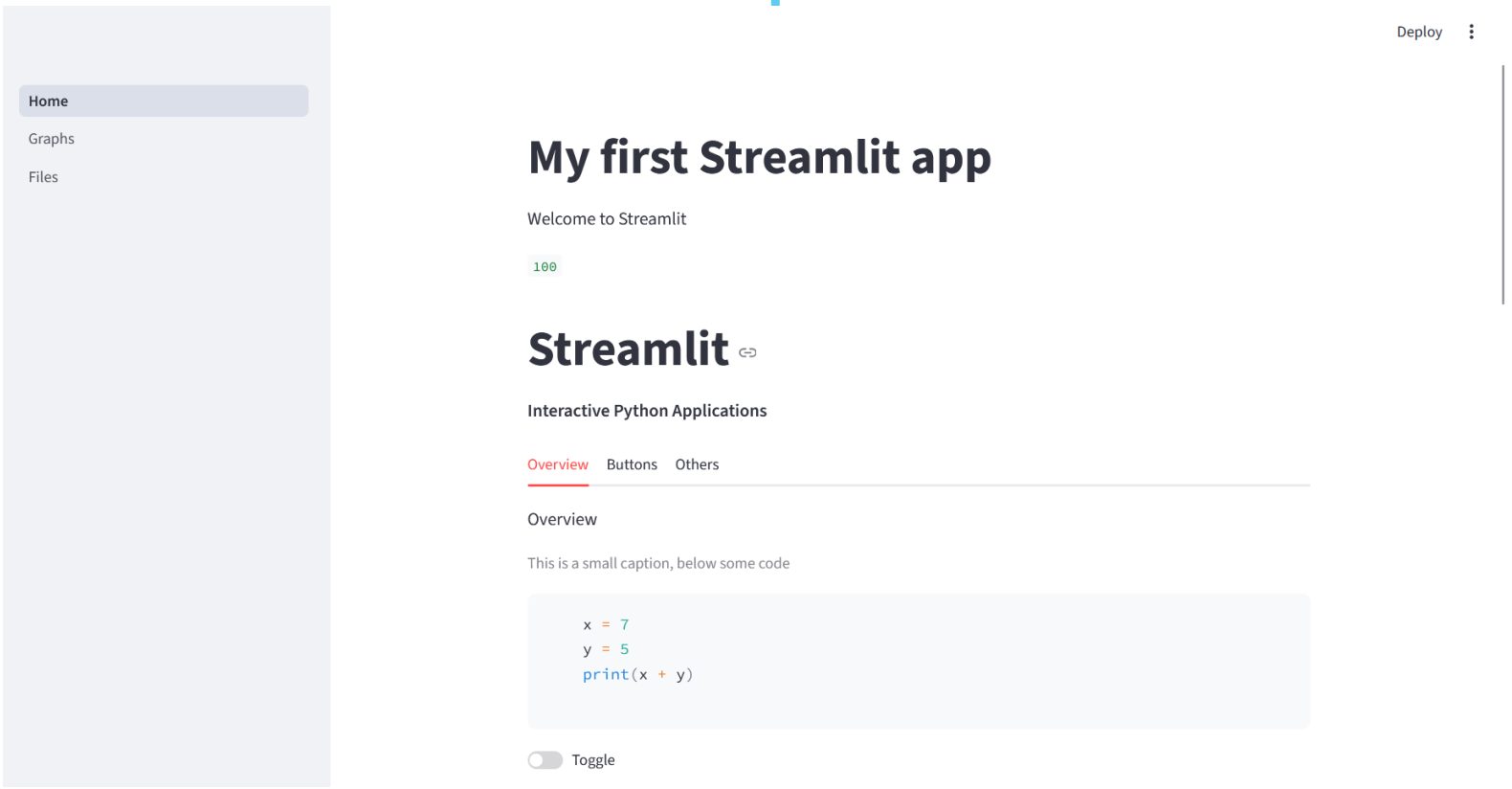
- library that converts scripts into web applications, focusing entirely on data-driven apps; documentation here
- based on widgets
- st.session_state is a built-in dictionary-like object that enables widgets and variables to retain their values even when the app is rerun; every interaction (a click, a keystroke in a widget, a slider move) reruns the whole script from top to bottom, thus is important to use st.session_state to perseve data

- to run the app: streamlit run your_script.py OR python -m streamlit run your_script.py

Streamlit - Commonly used widgets:

- **Text**: st.write, st.text, st.markdown, st.title, st.header, st.subheader, st.caption, st.code, st.latex, st.json, st.metric
- **Buttons**: st.button, st.download_button
- **Input Fields**: st.text_input, st.number_input, st.text_area
- **Sliders**: st.slider, st.select_slider
- **Selection Widgets**: st.selectbox, st.multiselect, st.checkbox, st.radio
- **File Uploaders**: st.file_uploader
- **Download**: st.download_button
- **Date/Time Pickers**: st.date_input, st.time_input
- **Progress and Status**: st.progress, st.spinner, st.balloons
- **Layout and Containers**: st.sidebar, st.columns, st.expander
- **Navigation:** st.navigation
- **Chat:** st.chat_message

Streamlit-Example



Lab6_app.py | Lab6.py | Lab6_pg2.py | Lab6_pg3.py

D: > 2-FACULTATE > Predat > FormalLanguagesAndCompilers > 2026 > Lab6_app

```python
import streamlit as st

pages = [
    st.Page("Lab6.py", title="Home"),
    st.Page("Lab6_pg2.py", title="Graphs"),
    st.Page("Lab6_pg3.py", title="Files")
]

pg = st.navigation(pages)

pg.run()
```

Lab6_app.py | Lab6.py | Lab6_pg2.py | Lab6_pg3.py

D: > 2-FACULTATE > Predat > FormalLanguagesAndCompilers > 2026 > Lab6.py > ...

```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


st.title("My first Streamlit app")
st.write("Welcome to Streamlit")
st.write(100)

# display text using Markdown formatting
st.markdown("# Streamlit")
st.markdown("**Interactive Python Applications**")

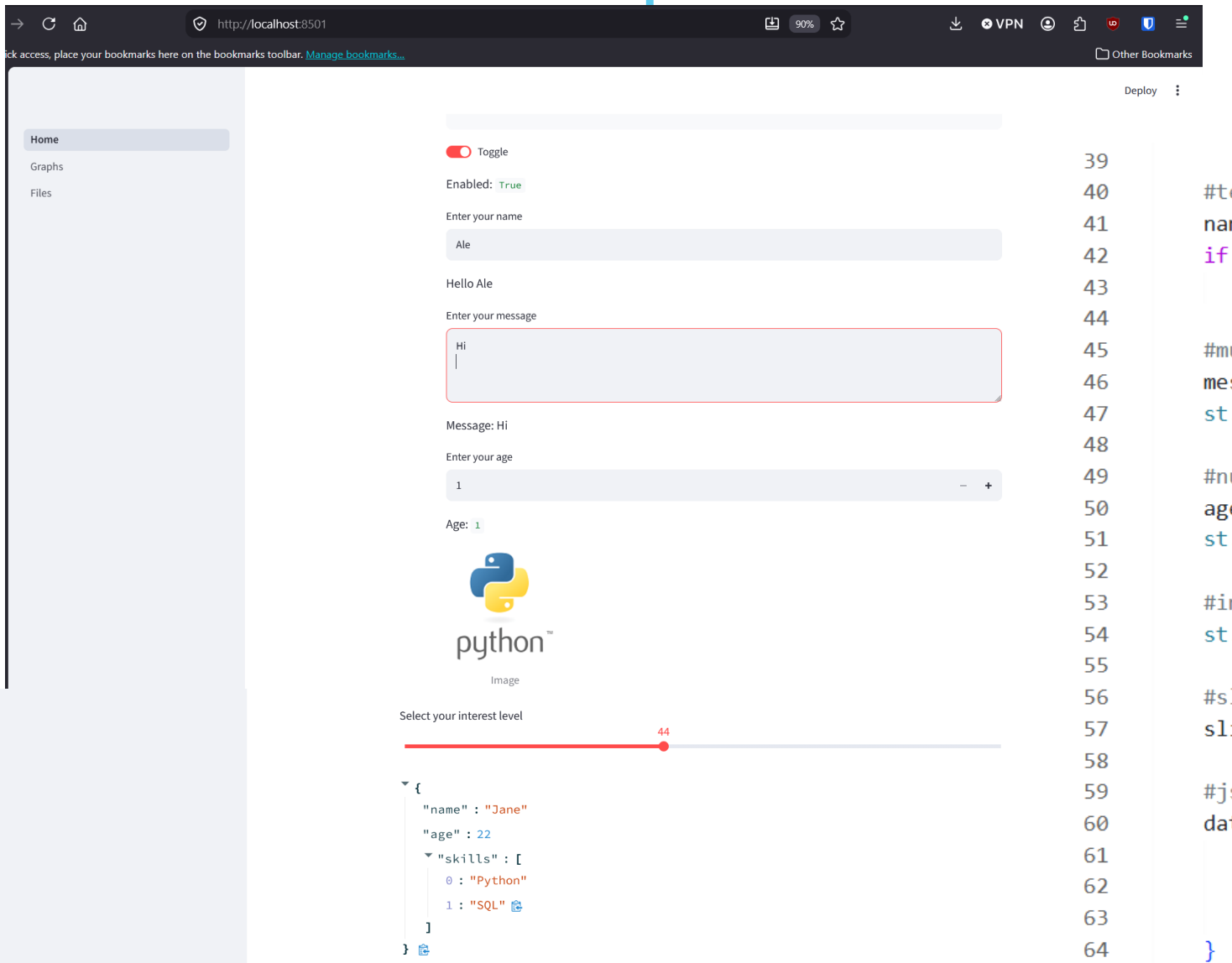

tab1, tab2, tab3 = st.tabs([
    "Overview",
    "Buttons",
    "Others"
])


with tab1:
    st.write("Overview")
    # display smaller supporting text
    st.caption("This is a small caption, below some code")

    #code
    code = """
    x = 7
    y = 5
    print(x + y)
    """
    st.code(code, language="python")

    #toggle
    tog = st.toggle("Toggle")
    st.write("Enabled:", tog)
```

Streamlit-Example




```
39
40      #text input
41      name = st.text_input("Enter your name")
42      if name:
43          st.write("Hello", name)
44
45      #multi-line text input
46      message = st.text_area("Enter your message")
47      st.write("Message:", message)
48
49      #number input
50      age = st.number_input("Enter your age", min_value=1, max_value=100)
51      st.write("Age:", age)
52
53      #image
54      st.image("Python-Symbol.png", caption="Image")
55
56      #slider
57      slid = st.slider("Select your interest level", 1, 100)
58
59      #json
60      data2 = {
61          "name": "Jane",
62          "age": 22,
63          "skills": ["Python", "SQL"]
64      }
65      st.json(data2)
66
67
```

Streamlit-Example

Streamlit

Interactive Python Applications

Overview | Buttons | Others

Buttons

Increment

Count: 0

Show Data

Choose a language

- Python
- Java
- C++

Selected: Python

Choose a city

Brasov

Selected city: Brasov

Choose your skills

SQL × Python ×

Selected Skills:

```
[
  0 : "SQL"
  1 : "Python"
]
```

```python
with tab2:
    st.write("Buttons")
    #button
    if "count" not in st.session_state:
        st.session_state.count = 0

    if st.button("Increment"):
        st.session_state.count += 1

    st.write("Count:", st.session_state.count)

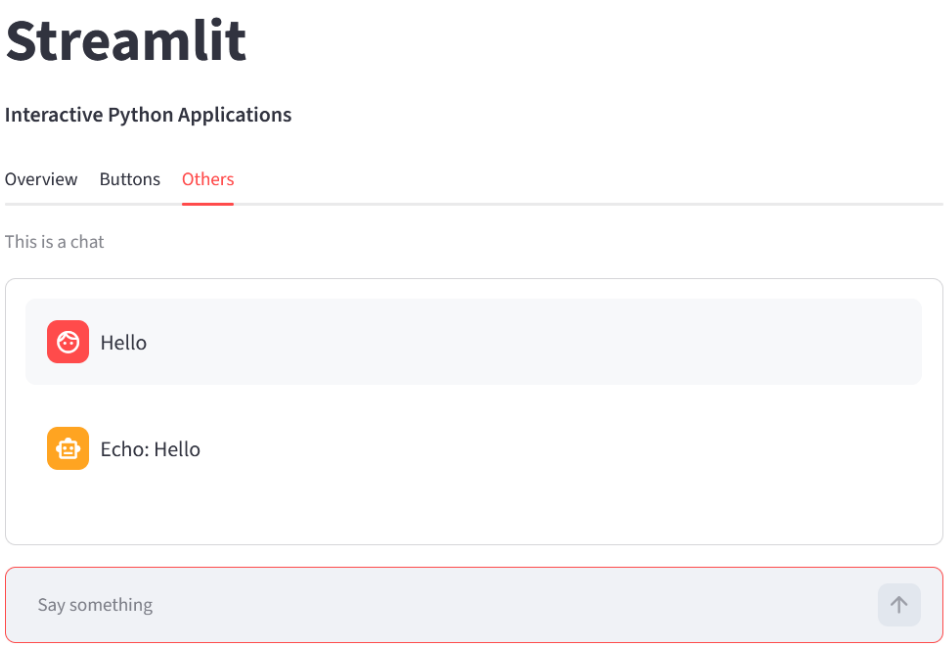
    #checkbox
    show_data = st.checkbox("Show Data")
    if show_data:
        st.write("Data is visible.")

    #radio button
    language = st.radio(
        "Choose a language",
        ["Python", "Java", "C++"]
    )
    st.write("Selected:", language)

    #dropdown selection
    city = st.selectbox(
        "Choose a city",
        ["Brasov", "Bucuresti", "Oradea"]
    )
    st.write("Selected city:", city)

    #select multiple options
    skills = st.multiselect(
        "Choose your skills",
        ["Python", "SQL", "Machine Learning", "Java"]
    )
    st.write("Selected Skills:", skills)
```

Streamlit-Example




```
105
106 with tab3:
107     st.caption("This is a chat")
108     #chat
109     messages = st.container(height=200)
110     if prompt := st.chat_input("Say something"):
111         messages.chat_message("user").write(prompt)
112         messages.chat_message("assistant").write(f"Echo: {prompt}")
113
```

Streamlit-Example

Home

Graphs

Files

Section 1

Streamlit Graphs

View Details

Additional information is displayed here.

DataFrame:

	Name	Marks
0	Jane	85
1	John	92
2	Joe	78
3	Katy	100

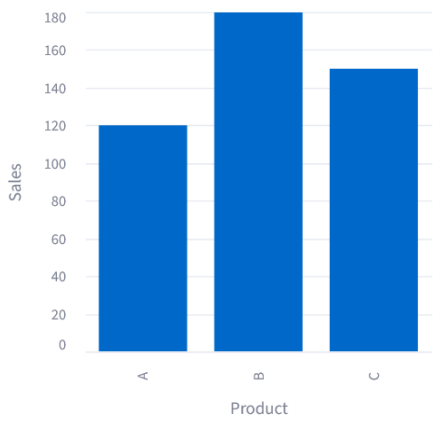
Table:

col 0	col 1	col 2	col 3	col 4
-0.8011	-0.0049	0.2104	-0.9360	-1.6399
-0.9904	0.8206	1.2587	-1.4055	-0.5614
0.5927	1.4290	1.8338	-0.3149	-0.8715
-0.1245	0.1476	0.8783	1.1521	1.2811
-1.3714	0.7454	0.5651	-0.3331	0.7945

Line chart:



Bar chart:



Lab6_app.py | Lab6.py | Lab6_pg2.py | Lab6_pg3.py

D: > 2-FACULTATE > Predat > FormalLanguagesAndCompilers > 2026 > Lab6_pg2.py > ...

```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

st.header("Section 1")
st.write("Streamlit Graphs")

with st.expander("View Details"):
    st.write("Additional information is displayed here.")

col1, col2 = st.columns(2)
with col1:
    #Dataframe
    data = {
        "Name": ["Jane", "John", "Joe", "Katy"],
        "Marks": [85, 92, 78,100]
    }

    df = pd.DataFrame(data)
    st.caption("DataFrame: ")
    st.dataframe(df)

    #line chart
    data3 = pd.DataFrame({
        "Sales": [100, 150, 130, 190, 80, 180, 200]
    })
    st.caption("Line chart: ")
    st.line_chart(data=data3, y_label='Sales', color="#ff006aa4")

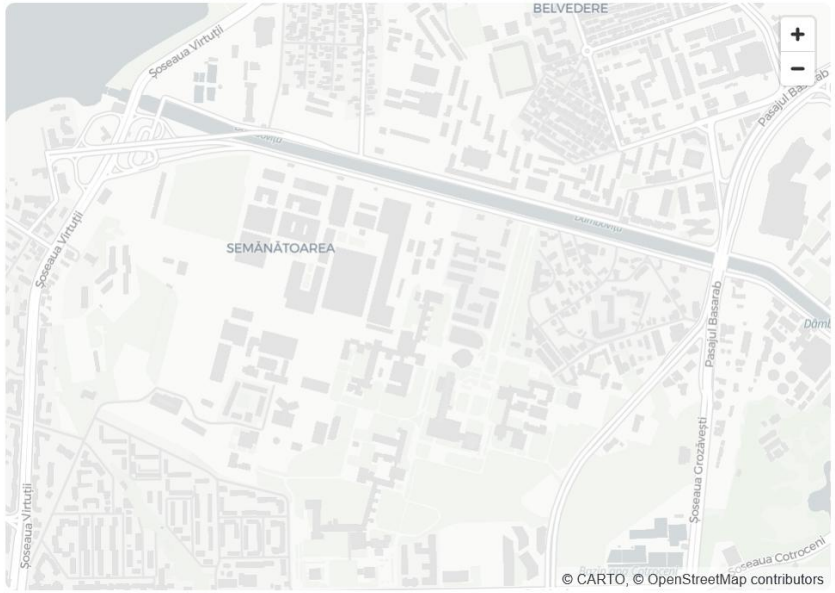
with col2:
    st.caption("Table: ")
    dataframe = pd.DataFrame(
        np.random.randn(5, 5),
        columns=('col %d' % i for i in range(5)))
    st.table(dataframe)

    #bar chart
    data4 = pd.DataFrame({
        "Product": ["A", "B", "C"],
        "Sales": [120, 180, 150]
    })
    st.caption("Bar chart:")
    st.bar_chart(data4, x="Product", y="Sales")

```

Streamlit-Example

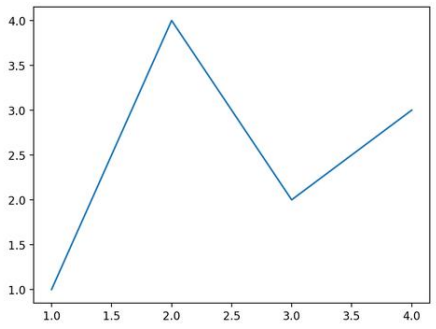
Map:



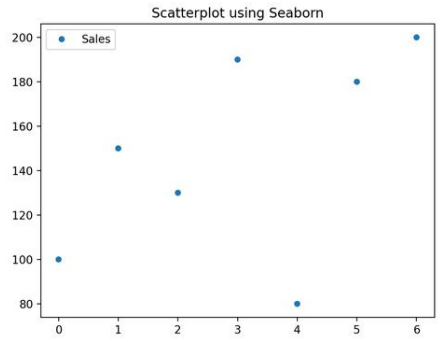
Section 2

Graphs from other libraries

Plot with Matplotlib



Plot with Seaborn




```
48
49  #map
50  data5 = pd.DataFrame({
51      "lat": [44.44220886342433],
52      "lon": [26.04954561978293],
53  })
54  st.caption("Map: ")
55  st.map(data5, zoom=14)
56
57  st.divider()
58
59  st.header("Section 2")
60  st.write("Graphs from other libraries")
61
62  col1, col2 = st.columns(2)
63
64  with col1:
65      #Plot with Matplotlib
66      fig, ax = plt.subplots()
67      ax.plot([1, 2, 3, 4], [1, 4, 2, 3])
68      st.caption("Plot with Matplotlib")
69      st.pyplot(fig)
70
71  with col2:
72      #StPlot with Seaborn
73      fig, a = plt.subplots()
74      a= sns.scatterplot(data=data3)
75      st.caption("Plot with Seaborn")
76      plt.title("Scatterplot using Seaborn")
77      st.pyplot(fig)
```

Streamlit-Example

Home
Graphs
Files

Deploy

Simple Data Explorer

Upload a CSV file

10 Sales Records.csv
0.7KB

Here's a preview of your data:

	Region	Country	Item Type	Total Revenue	Total Cost
0	Australia and Oceania	Tuvalu	Baby Food	2533654	1582243.5
1	Central America and the Caribbean	Grenada	Cereal	576782.8	328376.44
2	Europe	Russia	Office Supplies	1158502.59	933903.84
3	Sub-Saharan Africa	Sao Tome and Principe	Fruits	75591.66	56065.84
4	Sub-Saharan Africa	Rwanda	Office Supplies	3296425.02	2657347.52



Lab6_app.py | Lab6.py | Lab6_pg2.py | Lab6_pg3.py

D: > 2-FACULTATE > Predat > FormalLanguagesAndCompilers > 2026 > Lab6_pg3.py > ...

D:\

```python
import streamlit as st
import pandas as pd

st.title("Simple Data Explorer")

# Upload CSV
uploaded_file = st.file_uploader("Upload a CSV file", type=["csv"])

if uploaded_file is not None:
  df = pd.read_csv(uploaded_file)
  st.write("Here's a preview of your data:")
  st.dataframe(df.head())

  # Add a chart
  st.line_chart(df.select_dtypes(include=["number"]))

```

Streamlit-Example2

- https://studenttrackerexample.streamlit.app
- Code: https://github.com/DianaAlexandraC/FLC_Streamlit

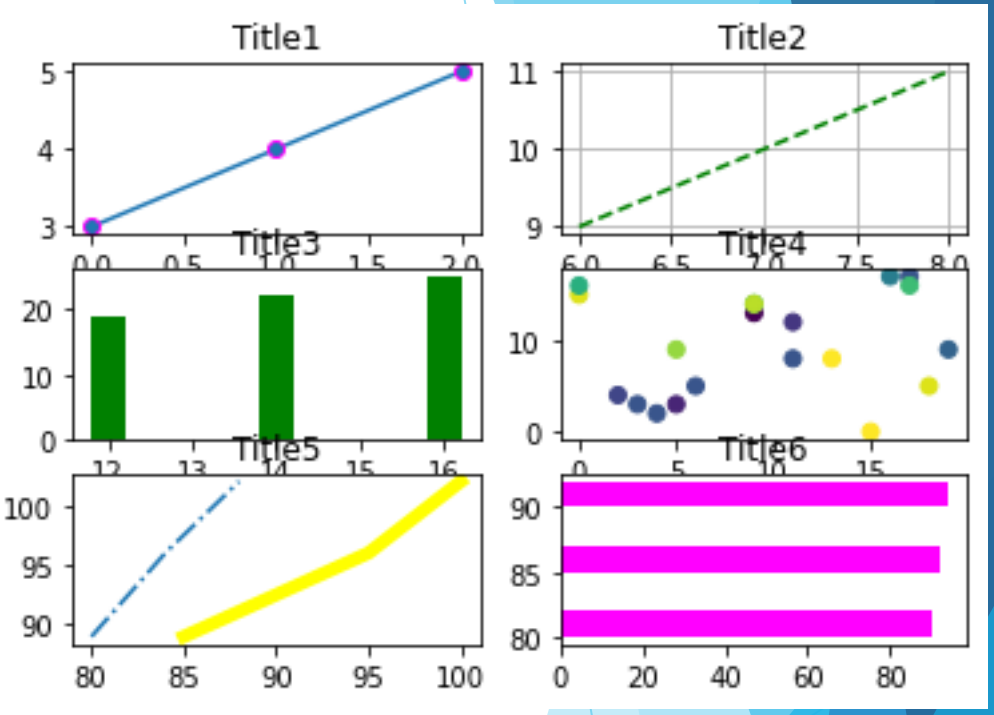
Exercises

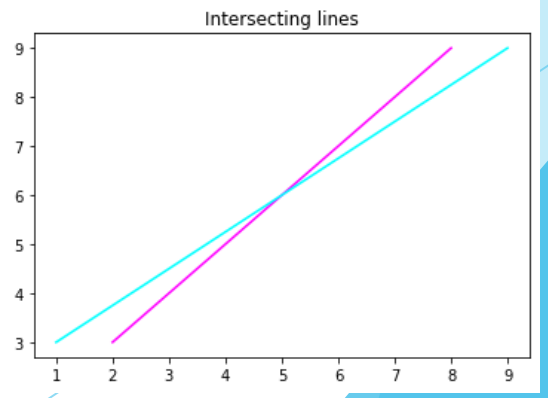
1. list1=[1.4,5.9,2.6,7.3,8.5,9.3,6.1] ; Convert it into an array using numpy. 0,5p
2. Write a program that creates a 1D array of even numbers between 70 and 91. 0,5p (Hint: use arrange() from numpy)
3. Write a program that creates a 3x3 matrix populated by 0s and having the numbers 3,7,11 on the main diagonal. (Hint: use diag() from numpy) 0,5p
4. Write a program that creates a 2x3 array of numbers between 10 and 16. (Hint: use arrange() and reshape() from numpy). 0,5p
5. ex5=np.array([[10,50,70,20,40],[5,45,95,35,65]]) ; Print all the numbers below and above 43 inside ex4 array. 0,5p
6. Use ex5 array. Write a program that saves the array in a text file, then load the content of it. 0,5p

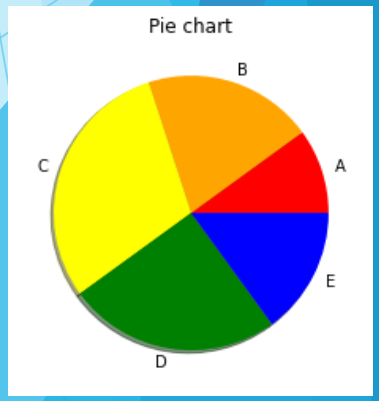
Exercises

7. Using matplotlib, create a 3x2 subplot with:
   a. Line with custom marker color for points 1p
   b. Line with custom style and grid 1p
   c. Vertical bars with custom color, width 1p
   d. Scattered points with different colors 1p
   e. 1 line with custom style, color & 1 line with custom width, color 1p
   f. Horizontal bars with custom color, height 1p
8. Create a plot with intersecting lines of different colors. 0,5p
9. Create a pie chart with 5 items of custom colors and shadow. 0,5p

!Use titles for all graphs







Homework

1. Using Seaborn and sns.load_dataset("tips"):
    a) Load the dataset. Print the first 5 rows, the shape, the number of rows for each day, and the summary statistics. (Hint: head(), shape, value_counts(), describe()) 1p
    b) Draw a histogram of total_bill, with one color per time and a density curve. (Hint: histplot() with hue and kde) 0,5p
    c) Draw a scatter plot of total_bill against tip, with a different color for each day. (Hint: scatterplot()) 0,5p
    d) Using plt.subplots(1, 2), draw in the same figure a box plot of total_bill per day and a violin plot of tip per day. (Hint: ax=ax[0] and ax=ax[1]) 1p
    e) Draw a heatmap of the correlations between the numeric columns, with the values written inside the cells. (Hint: heatmap() with annot=True, cmap="coolwarm") 1p
2. Using Streamlit:
    a) Display a title, a header, a text containing a bold word, a caption, and a code block with a short Python snippet. 1p
    b) Add a text input for your name, a slider for the study hours per week (0 to 40) and a select box with 3 courses. Below them, show a message built from the three values. 1p
    c) Create a counter with the buttons Increment and Reset. The value must be kept between reruns. (Hint: st.session_state) 1p
    d) Create with NumPy a DataFrame with 20 random scores (0 to 100) for 3 subjects. Show it with st.dataframe. In the sidebar, add a multiselect to choose the subjects, then show a line chart and a Seaborn box plot of the chosen subjects. (Hint: st.sidebar, st.line_chart, sns.boxplot, st.pyplot) 2p
    e) Add a CSV uploader. When a file is uploaded, show a preview and a line chart of its numeric columns. If no file is uploaded, show a message with st.info. (Hint: st.file_uploader, select_dtypes) 1p

Use titles for all graphs!